

It's not a lie if you don't get caught: simplifying reconfiguration in SMR through dirty logs

Allen Clement
Subzero Labs

Natacha Crooks
Subzero Labs
UC Berkeley

Neil Giridharan
Subzero Labs
UC Berkeley

Alex Shamis
Subzero Labs

Abstract

Production state-machine replication (SMR) implementations are complex, multi-layered architectures comprising data dissemination, ordering, execution, and reconfiguration components. Existing research consensus protocols rarely discuss reconfiguration. Those that do tightly couple membership changes to a specific algorithm. This prevents the independent upgrade of individual building blocks and forces expensive downtime when transitioning to new protocol implementations. Instead, modularity is essential for maintainability and system evolution in production deployments. We present Gauss, a reconfiguration engine designed to treat consensus protocols as interchangeable modules. By introducing a distinction between a consensus protocol's *inner log* and a sanitized *outer log* exposed to the RSM node, Gauss allows engineers to upgrade membership, failure thresholds, and the consensus protocol itself independently and with minimal global downtime. Our initial evaluation on the Rialo blockchain shows that this separation of concerns enables a seamless evolution of the SMR stack across a sequence of diverse protocol implementations.

1 Introduction

State-machine replication (SMR) [26] has emerged as the fundamental fault-tolerant core of modern distributed applications. Most database systems use crash-fault-tolerant SMR protocols such as Raft or Paxos to defend against data loss in the event of machine failures. Spanner, Google's geo-distributed flagship database uses Multi-Paxos at its core; Neo4j, a popular graph database, uses Raft, while Cassandra leverages Accord [1], a leaderless consensus protocol. Blockchain systems such as Rialo [25] or Sui [11] similarly derive their security guarantees from Byzantine-fault-tolerant (BFT) consensus protocols like PBFT [5], Hotstuff [31], Mysticeti [3] or Autobahn [12].

Consensus protocols have a hard job: they must offer strong safety and liveness guarantees at high throughput and low latency. Research in this space is rich [2, 3, 5–8, 12, 13, 17, 21, 22, 27]. Consensus *engineers*, however, have an even harder job! Not only must they deliver on the promises made by the protocol, but they must additionally face the

operational realities of modern distributed systems, both technical and socio-technical.

On the technical front, engineers must grapple with the realities of hardware degradation, network partitions and shifting load patterns. Whether it is a global database like Spanner migrating a shard between continents to follow user activity, permissioned blockchains like Sui or Rialo rotating its validator set to ensure forward security, or a graph database removing a misconfigured machine suffering from a fail-slow fault, the system must support the dynamic addition and removal of consensus participants. Unfortunately, academic research on consensus primarily assumes a fixed universe of participants, P , known a priori and immutable for the duration of the system's execution. In this idealised static model, the quorum size Q is constant, the failure threshold f is fixed, and the identity of every participant is hard-coded into the protocol state. A survey of recent consensus papers published at top security/systems venues show that only a small fraction had a detailed *reconfiguration protocol* to change the participant set.

On the socio-technical front, engineers must also navigate organisational changes, team restructurings, and evolving business requirements that impact system design and operation. There are two primary software engineering challenges: 1) modularity is essential for future-proofing 2) code evolves. Modularity, even inside of the consensus module, is key for maintainability. A realistic SMR implementation contains several components: the data dissemination layer, the ordering layer, the execution engine, and the reconfiguration logic among others. Each of these components should be modifiable and testable independently. Moreover, these building-blocks *evolve* in ways that are often unpredictable. A database or blockchain company rarely deploys a single consensus protocol. Instead, it deploys a sequence of consensus implementations. Each implementation improves over time as the company matures. Sui, for instance, a popular blockchain system, recently announced a "v2" of their internal consensus algorithm Mysticeti [29] while Cassandra recently migrated to the leaderless Accord CFT consensus protocol [10]. Overall correctness must not only hold across

a dynamic set of participants, but across a sequence of consensus protocol implementations! Unfortunately, all existing reconfiguration algorithms today tightly couple the reconfiguration logic with a *single* consensus protocol. There does not currently exist a clean way to update consensus. Incremental updates are hard enough; dramatic changes, including adoption of a new protocol entirely, are expensive and take years. Evolution usually results in protocol downtime and code bases often lose the ability to access old committed transactions when consensus changes.

In this paper, we argue that a practical reconfiguration protocol should achieve the following three properties: 1) arbitrary membership changes 2) full modularity 3) minimal downtime:

- **Arbitrary Membership Change** It should be possible to replace any and all participants from one configuration to the next. This consideration is especially important in blockchain systems where validators have stake associated with their votes.
- **Full Modularity** The reconfiguration logic should be strictly independent of all other components in the SMR node. It should be possible to transition the underlying consensus engine from one arbitrary consensus protocol to another, without knowing anything about the details, except for the fact that it satisfies the traditional safety and liveness definitions.
- **Minimal Downtime** We contend that the operational benefits of satisfying Goals 1 and 2 outweigh the benefits of achieving optimal performance during a reconfiguration event. Reconfiguration is traditionally rare. As such, we can tolerate brief latency spikes during the reconfiguration process as long as downtime remains minimal.

This paper presents the design and implementation of a novel reconfiguration engine, Gauss, that satisfies all three properties. The reconfiguration logic makes no assumption about the underlying consensus protocol used to order transactions, only that it satisfies the safety and liveness property of consensus. Yet it achieves full generality: two epochs i and $i+1$ can differ in their consensus protocol, membership, as well as failure thresholds. Gauss achieves this with minimal downtime.

To achieve this, we take as our starting point the ideas of Horizontal Paxos [19], a classic reconfiguration protocol for crash-fault-tolerant (CFT) systems. Each consensus's log within an epoch provides a global consistent order across replicas that can be leveraged to disseminate reconfiguration information. To achieve full generality, however, we make a second observation: the log of operations generated by the consensus protocol itself need not necessarily be the log exposed by the consensus engine to other components of the SMR node. We can lie! Specifically, a transaction marked as committed in a consensus protocol's *inner log* need not necessarily be marked committed in the *outer log* exposed

to other components, as long as the overall SMR properties are preserved. Fundamentally, this idea is not new. Transactions submitted to multiple leaders must be deduplicated in most leader-based systems, BFT or CFT. Blockchain systems check external validity predicates prior to executing the transaction.

Applied to reconfiguration, however, distinguishing the inner log and outer log in this way is powerful. It allows seamless transition between consensus epochs without making *any* assumptions about the protocols themselves. For instance, it achieves even greater generality than the original Horizontal Paxos algorithm: Gauss need not bound the number of concurrent/pending transactions in the system.

We implement this idea as part of the Rialo blockchain, cleanly modularising each RSM component for future upgradeability.

2 System Model

Practical deployments of state machine replication protocols must support 1) a changing set of participants 2) updates to the consensus protocol itself. Existing formalisms rarely consider this reality. We first introduce formalisms that capture the long-term, evolving nature of real SMR deployments that must continue to provide safety and liveness guarantees across a sequence of changing configurations and consensus protocols.

An SMR system must satisfy the following top-level properties:

- **Safety.** If two honest nodes commit transactions (j, x) and (j, x') , respectively, for the same log position j , then $x = x'$.
- **Liveness.** If an honest node proposes a transaction x , then all honest nodes eventually commit (j, x) for some log position j .
- **Integrity.** A transaction x appears in the log at most once. In particular, there do not exist two distinct log positions $j \neq j'$ such that honest nodes decide (j, x) and (j', x) .
- **External Validity.** For any committed transaction x , $\text{ExVal}(x) = \text{true}$, where ExVal is a predicate that checks whether x upholds all application invariants.

These are the properties that external clients observe. The SMR system can ingest this log to materialize shared state. Internally, however, the SMR system is composed of a sequence of epochs $e_1, e_2, \dots$. Each epoch e_i is associated with a membership configuration M_i , which specifies a set of n_i participating nodes, and a consensus protocol C_i . The protocol C_i exposes *propose*(x) and *decide*(j, x) events, where *decide*(j, x) indicates that transaction x is chosen for log position j . We assume that C_i satisfies the safety and liveness properties for consensus - it outputs a totally ordered log, but make no additional assumptions about the internal structure of C_i . Nodes

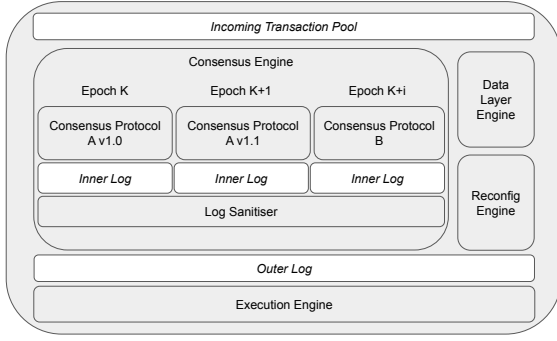


Figure 1. RSM Node SubComponents

in a configuration communicate over an asynchronous network that places no bounds on message delivery times. We assume that in an epoch e_i with n_i nodes, at most f_i nodes may fail during execution. Failures follow either the crash fault model or the Byzantine fault model. In the crash fault model, a faulty node may halt at any time but otherwise behaves correctly. In the Byzantine fault model, faulty nodes may behave arbitrarily. To tolerate f_i failures, we assume $n_i \geq 2f_i + 1$ under crash faults and $n \geq 3f_i + 1$ under Byzantine faults.

The system combines all of the inner logs to produce a final, sanitized, totally ordered log called the *outer log* that satisfies the aforementioned SMR properties. This layered structure captures the reality of modern SMR systems, where the consensus protocol is but one component in a larger architecture.

3 Design

In this section, we present the design of Gauss, our modular architecture that supports the independent evolution of individual system components.

3.1 RSM Node Architecture

An RSM node typically consists of four key components (Figure 1): 1) a data dissemination layer, tasked with disseminating transaction data to all nodes in the system 2) a consensus engine, tasked with outputting an ordered log of transactions consistently across all RSM replicas 3) an execution engine, tasked with ingesting this log to materialize the final application state 4) the reconfiguration engine, tasked with upgrading the different components of the system. These components work closely together to maintain the correctness of the replicated state machine. In a production implementation of an RSM node, these units must be designed in a modular fashion such that they can be independently modified and upgraded over time. This requires careful interface design, and clear separation of concerns at each component boundary. In this paper, we focus on the design of the reconfiguration engine, responsible for coordinating the transition between epochs.

3.2 Reconfiguration Engine

In line with our stated objective, our reconfiguration engine, Gauss, makes no assumption about the underlying consensus protocol used to order transactions, only that it satisfies the safety and liveness property of consensus (Section 2). Yet it achieves full generality: two epochs i and $i+1$ can differ in their consensus protocol, membership, as well as failure thresholds. Gauss achieves this with minimal downtime, by leveraging two observations: 1) a given consensus protocol within an epoch already outputs a totally ordered *inner log* of transactions across all correct nodes in the epoch 2) this *inner log* need not be directly executed, instead, it can be sanitized and transformed to produce a new *outer log*. Only the outer log is exposed to the other node subcomponents.

The reconfiguration engine runs a three stage protocol ensuring a safe (and live) transition from one epoch configuration to the next. The *prepare phase* ensures that members of the new configuration prepare their local state and components to begin processing transactions. The *handover phase* coordinates between the two epoch configurations to seamlessly transfer control and establish a trust chain between each configuration. This trust chain provides evidence to external users that the previous configuration agrees and supports the next configuration. Finally, the *shutdown phase* ensures that the outgoing members of the old configuration can safely wind down while preserving liveness. The log sanitizer component at each replica regulates the translation from the consensus inner logs to the outer log exposed to other components of the RSM node.

We next describe each phase in turn, illustrating the transition between epochs i and $i+1$. Epoch i uses consensus protocol C_i with membership M_i and failure threshold f_i , while epoch $i+1$ uses consensus protocol C_{i+1} with membership M_{i+1} and failure threshold f_{i+1} . Consensus protocol C_i produces inner log L_i , while C_{i+1} produces inner log L_{i+1} . By definition of consensus (Section 2), all correct nodes in epoch i agree on the same log L_i , and all correct nodes in epoch $i+1$ agree on the same log L_{i+1} . Similarly, if a correct node sees T_i at position i in the log, all correct nodes eventually see T_i at position i in the log. For simplicity, we assign different identities to all nodes in M_i and M_{i+1} . We write $R_{(i,k)}$ to denote the k -th replica in epoch i . In practice, there may be significant overlap between configurations. As shown in Figure 1, nodes then simply run concurrent epoch instances.

Prepare Phase An operator submits an `EPOCHCHANGE()` transaction to consensus instance C_i . The logic by which the `EPOCHCHANGE()` transaction is generated and submitted is outside the scope of this paper. It may be generated by a trusted operator, via an on-chain governance mechanism, or as a result of a protocol update. This transaction is treated as any other transaction in the RSM node. By properties of consensus, it will eventually appear in the log of consensus C_i at some position K . The epoch change transaction consists

of the identity of the current epoch being transitioned from i , the new epoch being transitioned to e_{i+1} . The latter additionally contains the new configuration parameters: the membership M_{i+1} , failure threshold f_{i+1} , and consensus protocol C_{i+1} .

$R_{(i,k)}$: Detects EPOCHCHANGE(i, e_{i+1}) transaction

Upon commitment of the EPOCHCHANGE() transaction at position K in L_i , the log sanitizer of $R_{(i,k)}$ begins waiting for READY() messages from all members of epoch $i+1$. No other changes occur. Notably, the log sanitizer continues to process and output transactions from L_i to the outer log O for execution as normal.

$R_{(i+1,k)}$: Detects EPOCHCHANGE(i, e_{i+1}) transaction

Upon observing an EPOCHCHANGE() message, each incoming replica member $R_{(i+1,k)}$ of the new epoch $i+1$ synchronises its local components to begin processing transactions from epoch $i+1$. This includes initializing its consensus protocol C_{i+1} , ensuring that it is sufficiently up-to-date with the current outer log, as well as synchronising application and execution state. Once ready, it submits a READY($i, i+1, \text{hash}, \text{sig}_k$) transaction to C_i where i and $i+1$ denote respectively the epochs being transitioned from and to, hash is the hash of the EPOCHCHANGE() transaction, and sig_k is a signature of the READY() message. By properties of consensus, this transaction will eventually appear in the log of consensus C_i at some position j .

$R_{(i+1,k)}$: Submits READY() message as a transaction to C_i

Handover Phase Two possible cases arise: 1) all READY() transactions for epoch $i+1$ appear committed in the log before a new epoch change transaction is observed 2) a new epoch change transaction commits, preempting the initial epoch change. We discuss each scenario in turn. Crucially, every replica in epoch i can deterministically decide which case has occurred by examining the log L_i . By properties of consensus, all correct replicas in epoch i agree on the same log L_i .

Case 1: Successful Handover If all READY() transactions from members of epoch $i+1$ appear committed in the log before a new epoch change transaction is observed, $R_{(i,k)}$ considers the handover to be successful. It forms a *handover certificate* for epoch i , indicating that epoch i finished at position h in log L_i , where h is the log position of the last READY() transaction. The handover certificate contains the id of the old epoch i , information about the new epoch E_i , the position of the handover certificate in the log h , as well as the hash of the previous handover certificate for epoch i (hash_i). It signs the handover certificate and submits this as a DONE() transaction to consensus C_{i+1} . Replica $R_{(i,k)}$ continues participating in protocol C_i as normal.

$R_{(i,k)}$: Forms signed HANDOVER($i, E_{i+1}, h, \text{hash}_i$) certificate. Submits as transaction to C_{i+1}

Because the reconfiguration protocol makes no assumption about the underlying consensus protocol, it is possible that transactions continue to be committed in L_i after position h . This is true for multi-proposer systems [3, 12, 28], as well as systems that allow for parallel consensus instances [5]. Therein lies the power of combining horizontal reconfiguration with the inner/outer log distinction. By definition, all replicas output the same inner log. They consequently all form the same handover certificate, and stop processing the inner log of epoch i at the same position h in L_i . The log sanitizer of $R_{(i,k)}$ simply ignores all transactions at positions $h' > h$ in L_i and does not submit them to the outer log O . They will never appear committed to the overall SMR node. Submitting READY() messages to consensus increases latency for new configurations to become active, but is key to achieving minimal downtime. Transactions continue to be committed and executed in epoch i until the handover certificate is formed at position h . Only transactions after position h are discarded.

Case 2: Preemption Members of the new epoch may fail to reply, or fail to synchronize their state in a timely fashion such that they are ready to proceed in the new epoch. The protocol simply allows later epoch changes to preempt earlier ones. If a new EPOCHCHANGE() transaction appears in L_i before all READY() transactions from epoch $i+1$ have been committed, replica $R_{(i,k)}$ in epoch i considers the handover to have been preempted. It informs the log sanitizer to stop listening for READY() transactions from epoch $i+1$, and to instead listen for READY() transactions from members of the new epoch $i+2$. The protocol then proceeds as before.

Shutdown Phase The shutdown phase ensures that members of epoch i can safely wind down and terminate the epoch without violating safety or liveness. To achieve this, the protocol should make sure that 1) the new epoch is fully operational before old epoch members stop participating in the consensus protocol C_i 2) there is a single such epoch.

$R_{(i,k)}$: Detects $f_i + 1$ DONE($i, e_{i+1}, \text{hash}_i$) transactions in C_{i+1}

Upon detecting $f_i + 1$ DONE() transactions from distinct members of epoch i in consensus C_{i+1} , replica $R_{(i,k)}$ considers epoch $i+1$ to be fully active. At least one correct member of epoch i has submitted a DONE() transaction (there is a maximum of f_i malicious replicas), indicating that it has safely formed a handover certificate and submitted it to consensus C_{i+1} . The presence of an honest replica's DONE() transaction offers two guarantees: 1) all subsequent members can recover sufficient state (as the DONE() transaction includes a signed checkpoint and 2) the guarantee that no other epoch is active. Honest nodes would never send conflicting DONE() transactions. It can now safely stop participating in consensus protocol C_i and shutdown the epoch.

$R_{(i+1,k)}$: Detects $f_i + 1$ DONE($i, e_{i+1}, hash_i$) transactions in C_{i+1}

Upon detecting $f_i + 1$ DONE() transactions from members of epoch i in consensus C_{i+1} , replica $R_{(i+1,k)}$ considers epoch $i+1$ to be fully active. At least one correct member of epoch i has submitted a DONE() transaction (there is a maximum of f_i malicious replicas), indicating that it has safely formed a handover certificate and submitted it to consensus C_{i+1} . By properties of consensus, all correct members of epoch $i+1$ will see these $f_i + 1$ DONE() transactions at the same positions in L_{i+1} . The log sanitizer can now begin outputting committed transactions from L_{i+1} to the outer log O for execution. All other replicas in epoch $i+1$ proceed similarly.

3.3 Worked Example

We provide a short worked example to further explain our approach. Consider epochs k and $k+1$ as shown in Figure 2. Epoch k uses consensus protocol C_k (Mysticeti) with membership $M_k = \{R_{(k,1)}, R_{(k,2)}, R_{(k,3)}, R_{(k,4)}\}$ and failure threshold $f_k = 1$, while epoch $k+1$ uses consensus protocol C_{k+1} (PBFT) with membership $M_{k+1} = \{R_{(k+1,5)}, R_{(k+1,6)}, R_{(k+1,7)}, R_{(k+1,8)}\}$ and failure threshold $f_{k+1} = 1$.

Transactions T_1, T_2 commit in inner log L_k at positions 1 and 2. The log sanitizer forwards them to the outer log O . At position 3, an epoch change transaction is initiated, transitioning the system from epoch k to $k+1$ with a disjoint set of validators. The log sanitizer begins waiting to see READY() transactions appear in L_k . While waiting for these transactions, T_3 and T_4 commit in L_k and the log sanitizer forwards them to O . Upon detecting the final READY() transaction at position 9 in the log, the node forms a handover certificate finalising the epoch. It submits this certificate to C_{i+1} in epoch $k+1$ and discards transactions T_5 and T_6 (Figure 2). Upon seeing two DONE() messages, epoch $k+1$ becomes active. T_5 is resubmitted by a client and commits at position 3 of inner log L_{k+1} . Transactions T_7 and T_8 commit at positions 4 and 5 of inner log L_{k+1} . All are forwarded to the outer log for execution. The outer log exposes a single totally ordered log consisting of transactions $T_1, T_2, T_3, T_4, T_5, T_7, T_8$ and a trust chain of epoch transitions.

4 Proofs

We provide short proof sketches describing how our reconfiguration engine Gauss preserves the SMR properties of safety and liveness during epoch transitions for arbitrary consensus protocols and membership configurations.

Theorem 1 (Safety). *If two honest nodes decide (j, x) and (j, x') for the same outer log position j , then $x = x'$.*

Proof. Suppose, for contradiction, that $x \neq x'$. Let N_1 and N_2 be two honest nodes such that N_1 decides (j, x) and N_2 decides (j, x') . Each outer log position corresponds to an inner log position in some epoch. Let k denote the inner log

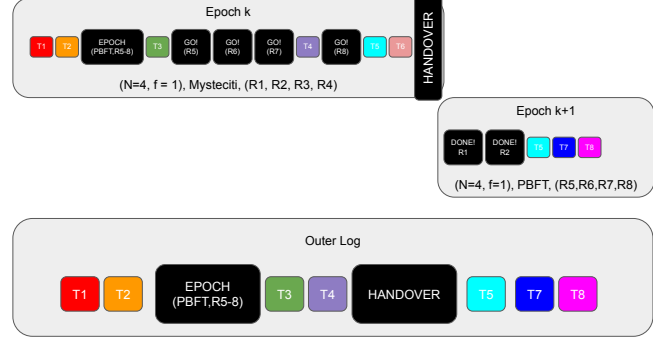


Figure 2. Epoch Transition Example

position corresponding to outer position j , and e the corresponding epoch. For each epoch i , let L_i and L'_i denote the inner logs produced by epoch i as observed by N_1 and N_2 , respectively. Let $L_e^{\leq k}$ (resp., $(L'_e)^{\leq k}$) denote the prefix of L_e (resp., L'_e) truncated to its first k entries.

Node N_1 computes x by applying the deterministic log sanitizer S to the concatenation $L_1 || L_2 || \dots || L_{e-1} || L_e^{\leq k}$, and similarly, N_2 computes x' by applying S to $L'_1 || L'_2 || \dots || L'_{e-1} || (L'_e)^{\leq k}$.

By the safety property of each consensus protocol C_i , all honest nodes agree on the contents of the inner log for epoch i . Therefore, for all $i < e$, we have $L_i = L'_i$, and moreover $L_e^{\leq k} = (L'_e)^{\leq k}$. Since S is deterministic, it follows that $x = x'$, contradicting our assumption.

Hence, $x = x'$, completing the proof. $\square$

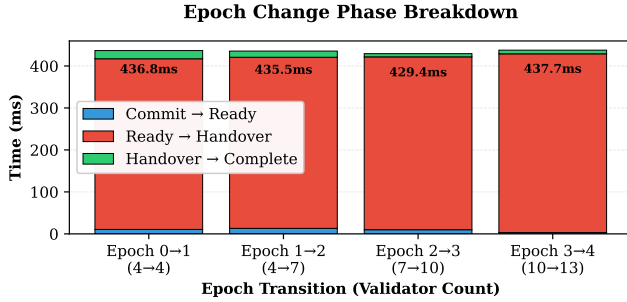
5 Implementation and Evaluation

This modular architecture is being implemented as part of the Rialo blockchain [25]. It provides the starting point for the seamless integration of future upgrades to the consensus engine, execution engine or data dissemination layer.

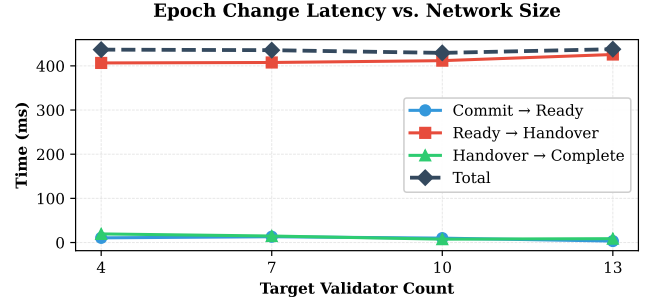
Initial results are promising. We evaluate the performance of our reconfiguration protocol on a local testbed to measure the latency of reconfiguration operations. We measure the time required to complete epoch transitions with varying validator set sizes. We test Rialo with 4 epoch transitions: from 4 to 4 validators (no size change), 4 to 7 validators, 7 to 10 validators, and 10 to 13 validators.

Figure 3a shows the performance characteristics of epoch transitions. We break down each transition into three phases: (1) from EpochChange commit to Ready message, (2) from Ready message to Handover (when the Ready quorum is reached), and (3) from Handover to completion.

The results show that the epoch change protocol performs efficiently across all tested configurations. We observe that the size of the validator set has a minimal impact on the overall latency of epoch transitions. The Ready to Handover phase takes approximately 93% of the total epoch change latency. This shows the commit of Ready messages in consensus is the main bottleneck. We expect that this would further



(a) Breakdown of each transition showing time spent in each stage.



(b) Latency scaling with validator count.

increase when we deploy the protocol in a geo-distributed setting.

6 Related Work

Reconfiguration in state machine replication has been carefully studied, both in the context of crash fault tolerance [15, 16, 18, 20, 24, 30] and Byzantine fault tolerance [4, 9, 14]. Unfortunately, the majority of consensus algorithms today do not explicitly discuss reconfiguration. Those that do so propose algorithms that are specific to a particular protocol, and thus tightly coupled with it.

CFT Reconfiguration. Classical state machine replication protocols like Paxos and Raft handle reconfiguration by treating configuration changes as special log entries, that must themselves go through consensus. The key challenge comes from handling the transition from one configuration to another. The work discusses a number of strategies to achieve that, often halting progress or limiting availability during the transition. Gauss is able to achieve identical results without limiting pipelining at the cost of delaying reconfiguration taking effect for additional consensus rounds. In SMART [20], reconfiguration of the system is managed by creating an additional group of replicas. The two groups of replicas run parallel Paxos instances until the system state is fully migrated to the new group. The popular *Raft* [24] consensus protocol initially proposed a joint-consensus approach where majorities from both old and new configurations must overlap, later refining this to single-server membership changes to simplify safety.

Vertical approaches to reconfiguration [18] decouple the configuration from the consensus protocol. Vertical Paxos, for instance, allows the set of acceptors to change within a single instance by relying on an auxiliary *configuration master* to manage membership. This “vertical” shift enables reconfiguration without stopping the stream of commands but introduces a dependency on an external, highly available master. Matchmaker Paxos [30] generalizes Vertical Paxos by replacing the auxiliary master with a set of “matchmakers.” These matchmakers persist configuration data, allowing the system to reconfigure without a stall in the pipeline or a single point of failure.

BFT Reconfiguration. BFT-SMART [4] extends the ideas of Horizontal Paxos to the Byzantine setting, allowing replicas to be added or removed one at a time through a special reconfiguration command. Duan et al. [9] offers a formal treatment of BFT with dynamic membership - and highlights the different possible optimisations as a function of assumptions on the system, including assuming a fraction of correct replicas never leave the system. BChain takes a different approach called rechainning for BFT protocols based on chain-replication. When a BFT service experiences failures or asynchrony, the head reorders the chain when a replica is suspected to be faulty, so that a fault cannot affect the critical path.

Permissionless Blockchains Permissionless blockchain systems such as Ethereum require that the consensus protocol remain live even if 1) a large number of participants is offline 2) the system has high churn rates, where the set of participants changes frequently. Neu et al. [23] explores the theoretical limits of consensus under dynamic availability, where nodes can join or leave arbitrarily. In this orthogonal setting, it identifies necessary conditions for safety and highlights that existing protocols often make unrealistic assumptions such as requiring synchrony.

7 Conclusion

This paper introduced Gauss, a novel reconfiguration protocol that supports both arbitrary membership changes and updates to consensus, while remaining fully modular. By distinguishing between the inner and outer logs, Gauss allows for seamless transitions between different consensus implementations without tightly coupling the reconfiguration logic to any specific protocol.

References

- [1] Karolos Antoniadis, Julien Benhaim, Antoine Desjardins, Elias Poroma, Vincent Gramoli, Rachio Guerraoui, Gauthier Voron, and Igor Zablotchi. Leaderless consensus. *Journal of Parallel and Distributed Computing*, 176:1–19, 2023.
- [2] Diogo S. Antunes, Afonso N. Oliveira, André Breda, Matheus G. Franco, Henrique Moniz, and Rodrigo Rodrigues. Alea-bft: Practical asynchronous byzantine fault tolerance. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2024.

- [3] Kushal Babel, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Arun Koshy, Alberto Sonnino, and Mingwei Tian. Mysticeti: Reaching the latency limits with uncertified dags. In *Network and Distributed System Security (NDSS) Symposium*. The Internet Society, 2025.
- [4] Alysson Bessani, João Sousa, and Eduardo E. P. Alchieri. State machine replication for the masses with bft-smart. In *Proceedings of the 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 355–366, 2014.
- [5] Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In *Proceedings of the 3rd Symposium on Operating Systems Design and Implementation (OSDI)*, pages 173–186. USENIX Association, 1999.
- [6] Allen Clement, Edmund Wong, Lorenzo Alvisi, Mike Dahlin, and Mirco Marchetti. Making byzantine fault tolerant systems tolerate byzantine faults. In *Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2009.
- [7] James Cowling, Daniel Myers, Barbara Liskov, Rodrigo Rodrigues, and Liuba Shrira. Hq replication: A hybrid quorum protocol for byzantine fault tolerance. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2006.
- [8] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. Narwhal and tusk: A dag-based mempool and efficient bft consensus. In *Proceedings of the 2022 European Conference on Computer Systems (EuroSys)*, 2022.
- [9] Sisi Duan and Haibin Zhang. Foundations of dynamic bft. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 1317–1334, 2022.
- [10] Apache Software Foundation. Cep-15: Fast general purpose transactions (accord). <https://cwiki.apache.org/confluence/display/CASSANDRA/CEP-15%3A+Fast+General+Purpose+Transactions>, 2023. Accessed: 2026-01-30.
- [11] Sui Foundation. Sui: A next-generation smart contract platform. <https://www.sui.io/>, 2026. Accessed: 2026-01-30.
- [12] Neil Giridharan, Florian Suri-Payer, Ittai Abraham, Lorenzo Alvisi, and Natacha Crooks. Autobahn: Seamless high-speed bft. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP)*, 2024.
- [13] Guy G. Gueta, Ittai Abraham, Shelly Grossman, Dahlia Malkhi, Benny Pinkas, Michael Reiter, Dragos Seredinschi, Orr Tamir, and Alin Tomescu. Sbft: A scalable and decentralized trust infrastructure. In *Proceedings of the 40th IEEE Symposium on Security and Privacy (Oakland)*, 2019.
- [14] Heidi Howard, Fritz Alder, Edward Ashton, Amaury Chamayou, Sylvan Clebsch, Manuel Costa, Antoine Delignat-Lavaud, Cédric Fournet, Andrew Jeffery, Matthew Kerner, Fotios Kounelis, Markus A. Kuppe, Julien Maffre, Mark Russinovich, and Christoph M. Wintersteiger. Confidential consortium framework: Secure multiparty applications with confidentiality, integrity, and high availability. *Proc. VLDB Endow.*, 17(2):225–240, October 2023.
- [15] Leander Jehl and Hein Meling. Asynchronous reconfiguration for paxos state machines. In *Proceedings of the 15th International Conference on Distributed Computing and Networking (ICDCN)*, 2014.
- [16] Leander Jehl, Roman Vitenberg, and Hein Meling. Smartmerge: A new approach to reconfiguration for atomic storage. In *Proceedings of the 29th International Symposium on Distributed Computing (DISC)*, 2015.
- [17] Ramakrishna Kotla, Lorenzo Dahlin, Allen Clement, Edmund Wong, and Mike Dahlin. Zyzzyva: Speculative byzantine fault tolerance. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, 2007.
- [18] Leslie Lamport, Dahlia Malkhi, and Lidong Zhou. Vertical paxos and primary-backup replication. In *Proceedings of the 28th ACM symposium on Principles of distributed computing*, pages 312–313, 2009.
- [19] Leslie Lamport, Dahlia Malkhi, and Lidong Zhou. Reconfiguring a state machine. *ACM SIGACT News*, 41(1):63–73, 2010.
- [20] Jacob R. Lorch, Atul Adya, William J. Bolosky, Ronnie Chaiken, John R. Douceur, and Jon Howell. The smart way to migrate replicated stateful services. *SIGOPS Oper. Syst. Rev.*, 40(4):103–115, April 2006.
- [21] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. The honey badger of bft protocols. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016.
- [22] Ray Neiheiser, Miguel Matos, and Luis Rodrigues. Kauri: Scalable bft consensus with pipelined tree-based dissemination and aggregation. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, 2021.
- [23] Joachim Neu, Javier Nieto, and Ling Ren. On the limits of consensus under dynamic availability and reconfiguration. *arXiv preprint arXiv:2510.03625*, 2025.
- [24] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pages 305–319, 2014.
- [25] Rialo. Rialo: The high-performance modular blockchain for distributed systems. <https://www.rialo.io/>, 2026. Accessed: 2026-01-30.
- [26] Fred B. Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys*, 22(4):299–319, 1990.
- [27] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. Bullshark: Dag bft protocols made practical. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022.
- [28] Chrysoula Stathakopoulou, Tudor David Tudor, and Marko Vukolić. Mir-bft: High-throughput bft for blockchains. In *Proceedings of the 2nd Workshop on Scalable and Resilient Infrastructures for Distributed Ledgers (SRIDL)*, 2019.
- [29] Sui. Mysticeti: The next generation of sui consensus. <https://blog.sui.io/mysticeti-v2-sui-consensus/>, 2025. Accessed: 2026-01-30.
- [30] Michael Whittaker, Neil Giridharan, Joseph M Hellerstein, Heidi Howard, Ion Stoica, and Natacha Crooks. Matchmaker paxos: A reconfigurable consensus protocol. In *Proceedings of the 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 67–84, 2021.
- [31] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. Hotstuff: Bft consensus with linearity and responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC)*, 2019.